Sprint 2 Document

Team 52 — Clue! (G6046 Software Engineering)

1. General Information

Dates

Start Date	Sprint End Date
23/03/2026	09/04/2026

People Involved

Name	Role	Status
Thanh Shaw	Project lead for implementation: game loop, movement logic, console prototype, testing support	Active
Insik Ham	No identifiable submitted contribution beyond limited meeting attendance	Non-contributing
Omar Abu Isbeih	No identifiable submitted contribution beyond limited meeting attendance	Non-contributing
Murat Maimaiti	TBC	Non-responding
Shengyuan Liu	TBC	Non-responding

2. Sprint Goal

The main goal of Sprint 2 was to move from skeleton classes and early model work into a working text-based Clue! prototype. The game needed to run from a main method, deal cards, take turns, roll dice, move players around the board, support room entry, suggestions, disproving and accusations, and declare a winner correctly.

A second goal was to stabilise the underlying game model before any GUI work. This meant getting the board, movement rules, cards, murder envelope, player state and turn order working properly in code so that Sprint 3 could focus on JavaFX rather than basic rules implementation.

3. Tasks for this Sprint

Task	Assignee	Status
Implement Game class with playable turn order and core game loop	Thanh	Complete
Implement movement validation and board navigation rules	Thanh	Complete

Create text-based console prototype for playing the game	Thanh	Complete
Implement room entry, suggestion and disproving flow	Thanh	Complete
Implement accusation logic and win / elimination conditions	Thanh	Complete
Implement AI player using simple / random decision making	Thanh	Complete
Strengthen board, cards and game-state test coverage	Thanh	Complete
Produce design material for turn flow and core game classes	Team 52	Partial
Complete Sprint 2 documentation and review	Team 52	Partial

4. User Stories

- As a player, I want to roll dice so that I know how many squares I can move.
- As a player, I want to move my token around the board so that I can reach rooms.
- As a player, I want to enter a room so that I can make a suggestion.
- As a player, I want to make a suggestion so that other players must disprove it if they can.
- As a player, I want to make an accusation so that I can attempt to win the game.
- As a player, I want the game to tell me when I win or lose so that the game ends correctly.
- As a player, I want AI opponents so that the game can still be played with fewer human players.

5. Requirements to Fulfil in this Sprint

Functional Requirements

- FR7: Players take turns in a fixed order, roll dice and move the resulting number of squares.
- FR8: Players move horizontally or vertically only and cannot move through walls or blocked squares.
- FR9: When a player enters a room they may make a suggestion naming a suspect, weapon and the current room.
- FR10: Other players must show one matching card if they are able to disprove a suggestion.
- FR11: A player may make an accusation; if correct they win, if incorrect they are eliminated from taking further turns.
- FR12: The game supports at least one AI player using simple decision making.

Non-Functional Requirements

- NFR1: The code remains organised around OO principles with a clear model structure.
- NFR2: The console prototype compiles and runs reliably as a meaningful intermediate deliverable.
- NFR3: The implementation remains suitable for later GUI integration.

6. Design Notes

Architecture

Sprint 2 focused on the internal game model rather than presentation. The project remained centred around model classes such as Game, Board, Player, Dice, Deck and MurderEnvelope.

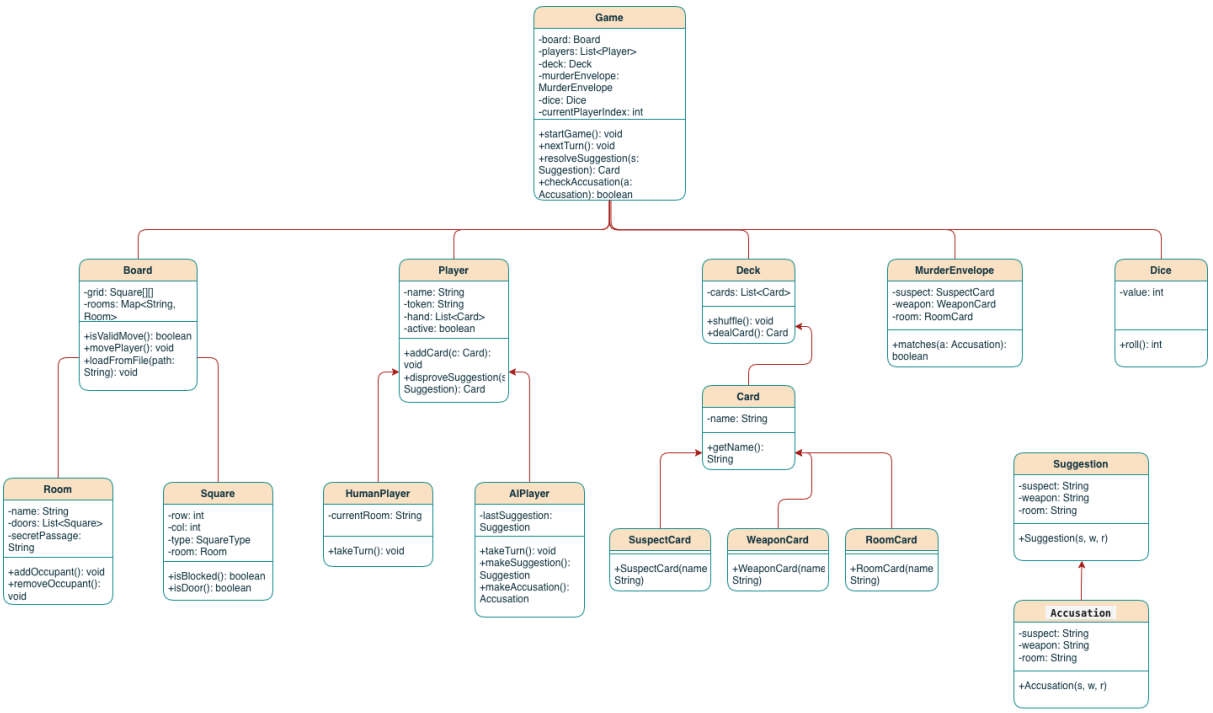
Class	Responsibility
Game	Coordinates setup, dealing, turn order, suggestions, accusations and overall state
Board	Stores board geometry, square types, room areas, doors and movement legality
Player / HumanPlayer / AIPlayer	Stores hand, token, location and player-specific turn behaviour
Deck / Card hierarchy	Creates and shuffles the 21 Clue! cards and supports dealing
MurderEnvelope	Stores the hidden suspect, weapon and room solution
Dice	Provides fair dice rolls for movement

Key Design Decisions

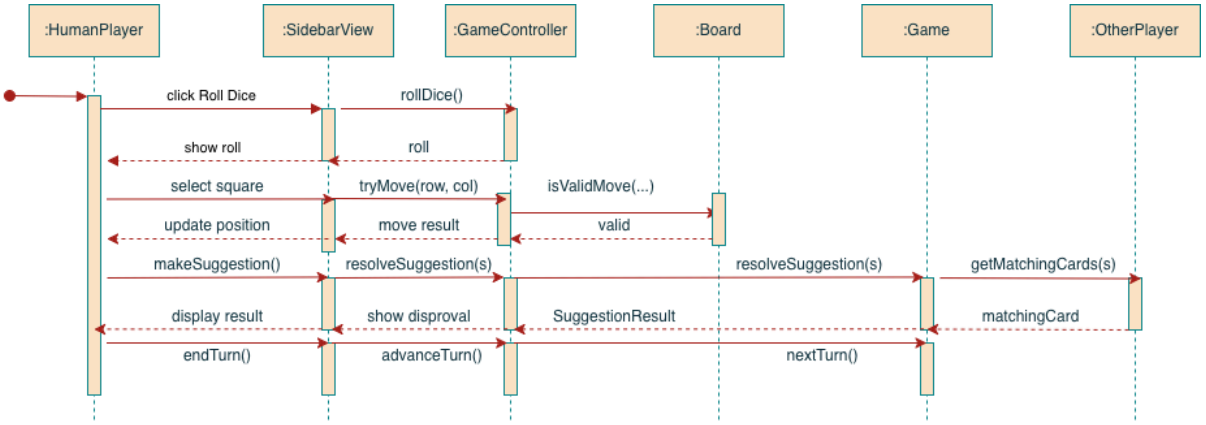
- Sprint 2 used a console interface deliberately so the rules and model could be validated before any GUI work.
- Movement validation was implemented against the board model rather than hardcoded into the turn loop.
- Suggestions were resolved by iterating through the other players in order until one could disprove.
- AI behaviour was kept simple and acceptable for the coursework brief, with more sophisticated behaviour deferred.
- The board and game-state logic were treated as reusable core systems so that a GUI could be added later without rewriting the rules layer.

Diagrams

Core game/model class diagram:



Normal Turn sequence diagram:



7. Testing Completed in this Sprint

Sprint 2 expanded unit testing and regression coverage around the working console prototype. By the end of the sprint, the project had a much stronger automated base than in Sprint 1.

Test Area	Coverage
Cards and deck logic	Correct card counts, card types, shuffling and dealing behaviour
Murder envelope	Correct hidden-card selection and accusation checking
Board and movement	Blocked squares, valid movement, room boundaries and start positions

Turn flow	Basic order of play, dice-driven movement and room entry
Suggestions and accusations	Disproving order, accusation correctness and elimination behaviour
Console / system testing	Manual playthroughs of the text-based prototype

Automated test coverage was increased to 48 JUnit tests by the end of Sprint 2. Console playtesting was used to validate end-to-end turn flow before GUI work began. The sprint exposed several logic issues that were easier to fix in the console prototype than they would have been later in JavaFX.

8. Sprint Changelog

- Moved from skeleton classes to a working text-based prototype.
- Implemented turn order and the main game loop.
- Added dice-driven movement and movement validation.
- Implemented room entry and room-based suggestion flow.
- Implemented suggestion disproving between players.
- Implemented accusation logic and win / elimination conditions.
- Added a basic AI player so the game could run with fewer human players.
- Expanded automated test coverage and stabilised the rules layer for later GUI work.

9. Post-Sprint Review

Did the sprint achieve its goal?

Yes. Sprint 2 delivered a working console-based Clue! prototype with recognisable gameplay. The game could be played through turns, movement, suggestions, disproving and accusations, which created a solid base for the JavaFX work in Sprint 3.

What went well?

- The team now had a working version of the game rather than just a set of classes.
- The console approach made it easier to debug the game rules before adding GUI complexity.
- The underlying model became much clearer and more stable by the end of the sprint.
- Automated testing improved significantly compared with Sprint 1.

What didn't go well?

- Team contribution remained uneven: most implementation progress still depended on one main contributor, while the other named members made no identifiable submitted contribution.
- Some planned design and documentation work was not completed alongside the coding work.
- The board and movement model still needed refinement and later required additional correction in Sprint 3.

What would we do differently?

- Keep design documentation and sprint records closer to the real implementation as it evolves.
- Share implementation and documentation workload more evenly where possible.

Carry-over to Sprint 3

- Replace the console prototype with a JavaFX GUI.
- Reuse the stable model layer rather than rewriting the rules.
- Improve usability, privacy handling and player feedback through the GUI.
- Continue expanding tests as more interaction logic is introduced.